

Path Planning using A*, RRT*, and Particle Filtering

Team #1

Ryosei Tanakamaru (Course 16)

J Desire Abayo (Course 6-3)

Matthew Kim (Course 6-2, 22, 2)

Franklin Chen (Course 6-3)

Robotics: Science and Systems

April 25, 2026

1 Introduction

Matthew Kim (Lab 6), Ryosei Tanakamaru (Lab 5) An autonomous vehicle navigating a known environment must continuously answer a deceptively simple question: *where am I?* Without a reliable answer, no higher-level behavior such as path planning, obstacle avoidance, or mission execution, is possible. Odometry alone cannot provide this answer — wheel-based dead reckoning accumulates error with every meter driven, causing position estimates to drift unboundedly over time. Additionally, a robot must also know *how* to get to its destination from its current position through a well defined path that avoids obstacles. Simply put, we cannot assume the robot to be able to avoid obstacles while also staying en route unless we instruct how to avoid obstacles and what routes it should stay on. This idea of avoiding obstacles while going from a start to end location is called path planning and involves the use of the previous lab to localize a robot within its environment.

Lab 5 addresses this foundational challenge through Monte Carlo Localization (MCL), implementing a full particle filter pipeline that fuses odometry and Light Detection and Ranging (LiDAR) to estimate the vehicle’s pose within a pre-built map (Stata Basement).

Lab 6 addresses this challenge through various possible path planning approaches that fall under sample-based and grid-based path planning combined with a pure pursuit algorithm to actually move the robot along the generated path. Using Lab 5’s localization, the robot interact with the real world to know what obstacles are nearby and what it should avoid within its path planning algorithm.

The Lab 5 component required the design and implementation of three interconnected components: a motion model, a sensor model, and the particle filter that integrates them. The motion model propagates a distribution of pose hypotheses forward in time by fusing odometry with bicycle model dynamics and injecting calibrated noise to capture actuator uncertainty. The sensor model evaluates each hypothesis against the known map using a precomputed range probability table derived from four weighted LiDAR failure modes, avoiding the per-scan raycast cost that would otherwise make real-time operation intractable. The particle filter combines both, iterating predict-weight-resample cycles until the population converges on the vehicle’s true pose.

The Lab 6 was broken into three parts: Part A involved planning a trajectory in a known occupancy grid map from the car’s current position to the goal pose, Part B involved programming the car to actually follow this predefined trajectory from Part A using the particle filter from Lab 5 and the pure pursuit control, and Part C involved combining the previous two parts to enable real-time path planning and execution. For Part A, our team used both a grid-based and sampling-based motion planning method to compare the two methodologies.

2 LAB 5 Technical Approach

Ryosei Tanakamaru

The lab's technical task was to publish the real-time position of the robot on a provided map, given an initial position guess, odometry data, and LiDAR data. We decided to approach this task using the Monte Carlo Localization. MCL is a particle filter algorithm that maintains a distribution of pose hypotheses and iteratively refines them using onboard sensor data. The pipeline has three stages, as shown in Figure 1.

First, the motion model propagates each particle forward in time by fusing odometry with bicycle model kinematics and injected noise, capturing the uncertainty inherent in wheel-based dead reckoning. Second, the sensor model scores each particle by comparing predicted LiDAR returns against the known map, assigning importance weights that reflect how well each hypothesis explains the observed scan. Third, the particle filter resamples the population toward the highest-weighted particles, publishes the weighted-average pose as the robot's estimated position, and feeds the updated distribution back into the motion model. Sections 2.1, 2.2, 2.3 detail each component.

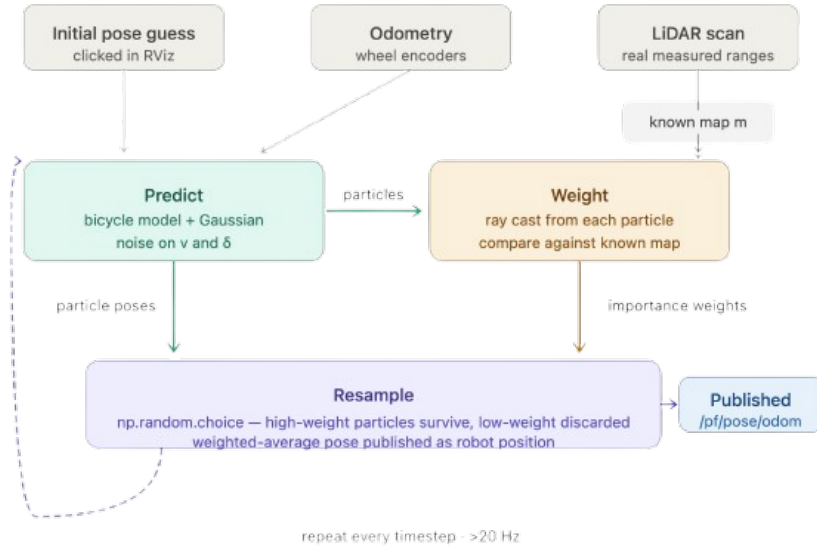


Figure 1: The Monte Carlo Localization pipeline. This shows how we combine odometry, initial guess and LiDAR scans to compute our current location.

2.1 Motion Model

Ryosei Tanakamaru

The motion model is responsible for propagating each particle's pose forward

in time given the robot’s measured motion. At each timestep it takes as input the current particle array — an $N \times 3$ matrix of (x, y, θ) hypotheses — and returns an updated array reflecting where each hypothesis would end up after the observed motion.

The base motion delta $(\Delta x, \Delta y, \Delta \theta)$ is taken directly from wheel encoder odometry. Rather than adding noise directly to the pose components, we inject uncertainty at the source: independent Gaussian perturbations σ_v and σ_δ are applied to the velocity and steering angle respectively, since these are the physical quantities most susceptible to error — particularly tire slip, which causes the actual heading rate to deviate from the commanded value. The perturbed inputs are then propagated through the bicycle model kinematics to produce correlated, physically consistent noise on $(\Delta x_i, \Delta y_i, \Delta \theta_i)$ for each particle:

$$\beta_i = \arctan\left(\frac{l_r}{L} \tan \delta_i\right) \quad (1)$$

$$\Delta x_i = v_i \cos \beta_i \cdot dt, \quad \Delta y_i = v_i \sin \beta_i \cdot dt, \quad \Delta \theta_i = \frac{v_i}{L} \cos \beta_i \tan \delta_i \cdot dt \quad (2)$$

where $v_i = v + \epsilon_v$, $\delta_i = \delta + \epsilon_\delta$, $\epsilon_v \sim \mathcal{N}(0, \sigma_v^2)$, and $\epsilon_\delta \sim \mathcal{N}(0, \sigma_\delta^2)$. This ensures that heading error from tire slip manifests as coupled uncertainty across all three pose components rather than independent additive noise on x , y , and θ separately.

The noisy pose increment for particle i is then represented as a transform matrix

$$\Delta T_i = \begin{bmatrix} \cos \Delta \theta_i & -\sin \Delta \theta_i & \Delta x_i \\ \sin \Delta \theta_i & \cos \Delta \theta_i & \Delta y_i \\ 0 & 0 & 1 \end{bmatrix} \quad (3)$$

and composed with the particle’s current pose matrix

$$T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & x_i \\ \sin \theta_i & \cos \theta_i & y_i \\ 0 & 0 & 1 \end{bmatrix} \quad (4)$$

via matrix multiplication to yield the updated pose

$$T'_i = T_i \cdot \Delta T_i \quad (5)$$

from which the new position and heading are extracted as $x'_i = T'_i[0, 2]$, $y'_i = T'_i[1, 2]$, and $\theta'_i = \arctan2(T'_i[1, 0], T'_i[0, 0])$. This batched $N \times 3 \times 3$ matrix formulation propagates all particles simultaneously in a single vectorized operation, which results in efficient computation.

2.2 Sensor Model

J Desire Abayo

The sensor model assigns an importance weight to each particle by computing the probability of the observed LiDAR scan z_k given the particle pose x_i and known map m . The full scan probability is the product of per-beam likelihoods across all n downsampled beams:

$$p(z_k|x_i, m) = \prod_{j=1}^n p(z_k^{(j)}|x_i, m) \quad (6)$$

Each per-beam likelihood is a weighted mixture of four distributions capturing distinct LiDAR failure modes:

$$p(z_k^j|x_i, m) = \sum_k \alpha_k p_k(z_k^j|x_i, m), \quad \sum_k \alpha_k = 1, \quad k \in \{\text{hit}, \text{short}, \text{max}, \text{rand}\} \quad (7)$$

p_{hit} models Gaussian noise around the true range d . p_{short} captures unexpected short returns from dynamic obstacles, nonzero only for $z < d$. p_{max} handles missed returns as a point mass at z_{max} . p_{rand} models spurious measurements as a uniform distribution over $[0, z_{\text{max}}]$.

Rather than evaluating this mixture at runtime, we precompute a 201×201 lookup table indexed by discretized true range d and measured range z . Simulated ranges from each particle are obtained via RangeLibC ray casting and used to index directly into the table, reducing per-beam cost to a single array lookup. Weights were tuned to $\alpha_{\text{hit}} = 0.74$, $\alpha_{\text{short}} = 0.07$, $\alpha_{\text{max}} = 0.07$, $\alpha_{\text{rand}} = 0.12$, and the full scan probability is raised to $\frac{1}{3}$ before resampling to prevent premature particle collapse.

2.3 Particle Filter

Franklin Chen

The particle filter integrates both the motion model and sensor together into the overall localization pipeline. In every state, the it maintains a constant population of N particles, each representing a pose hypothesis (x_i, y_i, θ_i) of the robot in the map frame.

At the start, after a pose estimate given by RViz, the particle filter initializes the localization process and samples N particles from a multivariate normal distribution around the clicked pose based on the pose’s covariance. This produces physically plausible particles without requiring global localization.

After each odometry measurement, the particle filter applies the motion model with an estimated motion to propagate every particle forward. The motion estimate is derived from the twist component of the odometry message, which represents the instantaneous velocity. After multiplying the twist by the elapsed

time and applying the kinematics of the bicycle model, the estimated motion is obtained.

After each LiDAR measurement, the sensor model evaluates how well each particle explains the observed measurements z , producing a corresponding weight vector w , where each w_i represents the probability of the particle i . These weights are squashed to reduce extreme disparities and normalized to sum to one. The particle filter then resamples the particles using `np.random.choice` with replacement, drawing N new particles according to the normalized weights. After the resampling, Gaussian noise are added to each particle to prevent premature collapse of the particles.

After either model update, the particle filter computes a pose estimate. The position is obtained by averaging particle positions, while the orientation is computed using circular mean:

$$\bar{\theta} = \arctan2\left(\sum_i \sin \theta_i, \sum_i \cos \theta_i\right) \quad (8)$$

which correctly handles angle wrapping. This estimate is published to `/pf/pose/odom`, and broadcast as a transform between the map frame and robot frame at each update.

3 LAB 5 Experimental Results

Ryosei Tanakamaru

We evaluated the particle filter in both simulation and on physical hardware in Stata Basement. Our primary performance metric is cross-track error, the perpendicular distance between the filter’s published pose estimate and the ground truth trajectory.

3.1 Simulation

Ryosei Tanakamaru

The particle filter was evaluated in simulation using the Stata Basement map loaded in RViz, with the car driven autonomously using the wall-follower from Lab 3. The filter was initialized at a clicked pose and the published estimate (`/pf/pose/odom`) was compared against the ground truth odometry (`/vesc/odom`) over the full trajectory. Figure 2 shows the simulation environment, and Figure 3 plots the estimates against ground truth. The filter tracked the ground truth closely throughout, achieving a mean cross-track error of 0.1102 m and a maximum of 0.3807 m. The maximum error occurred at sharp corners where the wall follower induced rapid heading changes, but the filter recovered quickly in all cases.

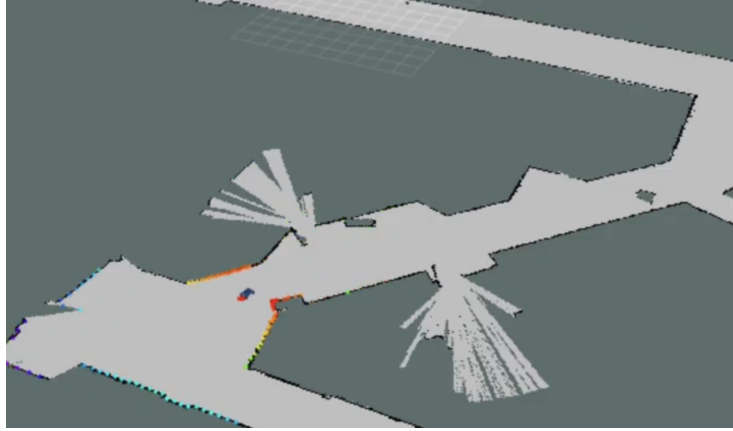


Figure 2: RViz simulation environment used for particle-filter evaluation, showing the Stata Basement map with the particles initialized around the clicked pose.

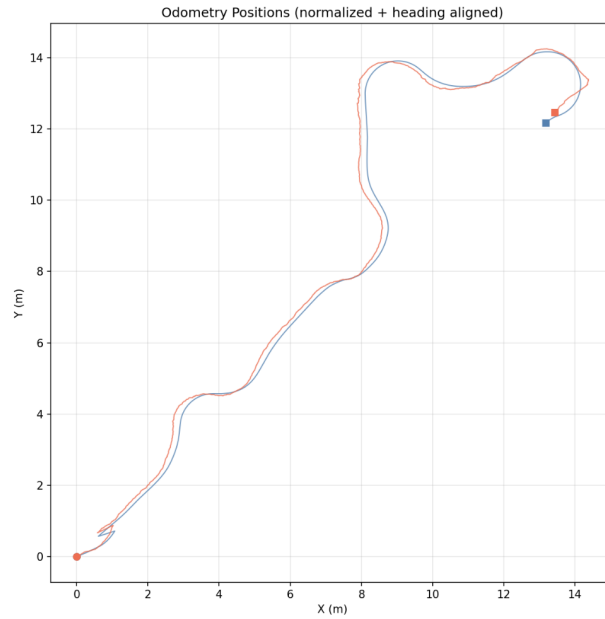
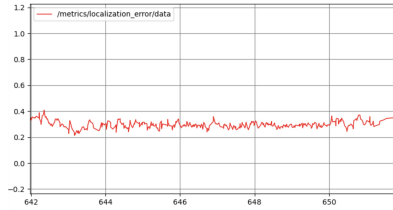
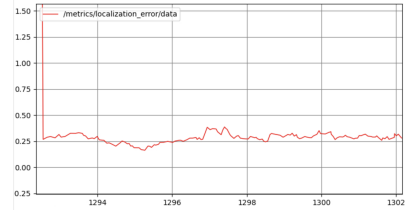


Figure 3: Quantitative comparison of the particle filter pose estimate vs. ground truth odometry over the simulation run. Orange is the ground truth and blue is the estimated positions. Mean cross-track error was 0.1102 m and maximum cross-track error was 0.3807 m, with spikes corresponding to sharp heading changes.

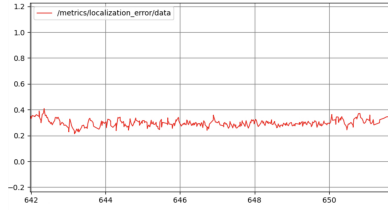
We additionally characterized the effect of particle count on filter performance. Convergence time, defined as the time until cross-track error fell below 0.3 m, scaled directly with particle count: 200 particles converged in 0.021 s, 500 in 0.041 s, and 1000 in 0.067 s. Steady-state cross-track error remained consistent across all three counts as shown in Figure 4, indicating that localization accuracy is bounded by sensor and map quality rather than particle count. This relationship motivated our choice of 200 particles as the operating configuration, yielding the fastest convergence without sacrificing accuracy.



(a) Steady state error with 200 particles



(b) Steady state error with 500 particles



(c) Steady state error with 1000 particles

Figure 4: Squared distance error vs. time (ROS time) for particle counts of 200, 500, and 1000. The plots show steady-state accuracy is consistent at approx. 0.25m across all configurations.

3.2 Real Experimentation

Matthew Kim

We evaluated the real life performance at three different noise levels and in two different maps. After additional conversations with the TAs, we are now presenting the testing runs involving the lowest level of noise since they are closest to the ideal scenario.

The first map is a straight path going through the STATA basement with Figure 5 being the RViz representation keeping 100 Odometry vectors and Figure 6 being the reconstructed positions with the blue `/vesc/odom` data and the red `/pf/pose/odom` representing the real and estimated paths respectively.

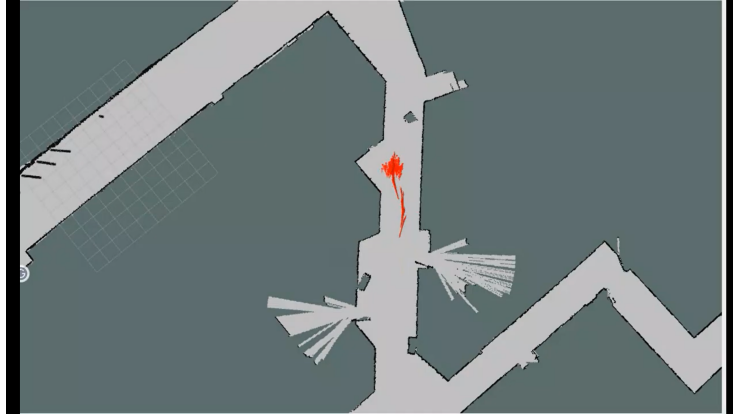


Figure 5: **Test 1: Linear Path.** Displays the particles as we drive the car in a linear path.

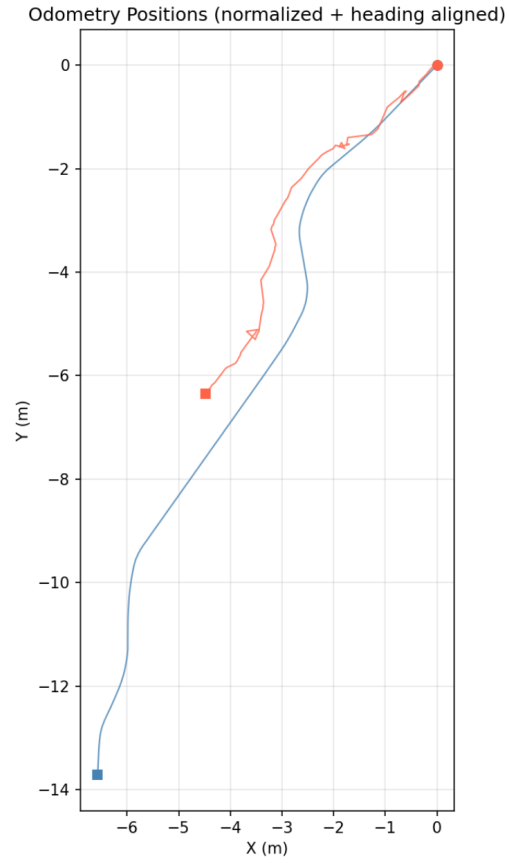


Figure 6: **Test 1: Linear Path.** Red line is our MCL estimate and blue is the car's odometry. The two lines show significant deviation.

The second map we used was going around the corner of the basement. Figures 7 and 8 have the same configurations as Figures 5 and 6 respectively.

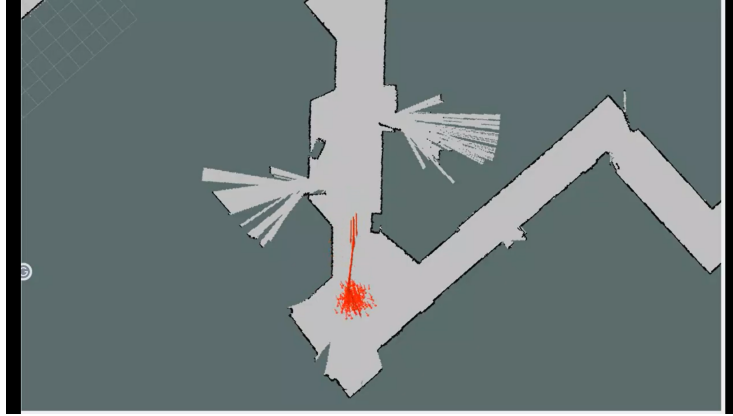


Figure 7: **Test 2: Turning Path.** Displays the particles as we drive the car.

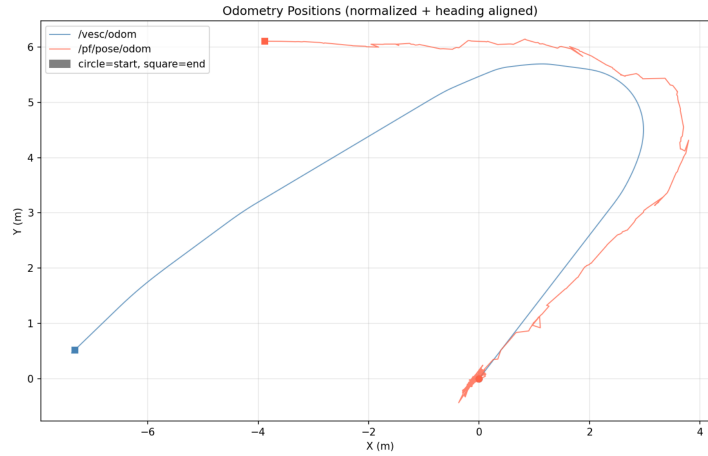


Figure 8: Test 2: Turning Path. Red line is our MCL estimate and blue is the car's odometry. This also shows significant deviation.

For the first map, we measured that the real life trajectory to be much longer (i.e. on the order of 10m+), so we expect the `/vesc/odom` data to be more accurate. The shorter length of the estimated path is suspected in part due to the hardware's latency issues and lag as well as any additional parameter tuning that is needed. The second map shows a similar trend of the `/vesc/odom` data being longer than the reconstructed pathway. The corner of the `/vesc/odom` data seems to be much more parallel than in real life, so we expect the real path to be in between the two paths. However, for all testing runs, the RViz representation appeared to be very accurate as we tracked the robot's path in real life, so we are hoping to fine tune our model and hardware to get a more

accurate reconstruction.

In future testing, we also plan to measure out the lengths of a few paths explicitly to determine which set of data is more accurate. Additionally, we ran into a bug where the LiDAR scan's origin would not accurately map onto the live position of the robot, which was unique to the real life implementation since it worked well in the simulation. This will be discussed further in section 4, but since the briefing, this issue has been fixed and may help fix some unseen bugs.

4 LAB 5 SLAM

Matthew Kim

SLAM stands for Simultaneous Localization and Mapping. It builds a map of a robot's environment while simultaneously tracking its environment by using sensors (like cameras or LiDAR sensors) to detect landmarks and calculate the robot's movement. This algorithm continuously updates the map and position via algorithms including loop closure, where the robot recognizes a previously visited location to correct any positioning errors that have built up over time, ensuring the map remains accurate and consistent, to correct for errors. For the purposes of the lab, we can see the Monte Carlo Implementation as mapping or matching the robot's sensory input to a known map while SLAM recreates the entire map from complete scratch. To explain in more detail, SLAM compares to Monte Carlo Localization (MCL) as a more comprehensive, complex process; SLAM creates maps, while MCL (a particle filter) only localizes a robot within a pre-existing map.

Using the provided SLAM algorithm and the visualization and observability platform Foxglove, we were able to recreate the following corner of the STATA basement with our physical robot.

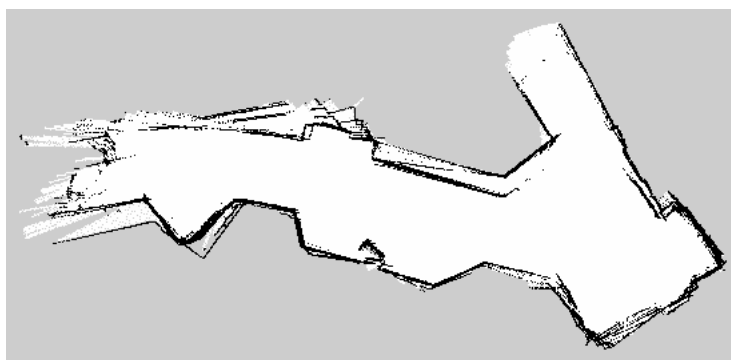


Figure 9: SLAM Recreation of STATA Basement Corner

After passing by the same area multiple times, the loop closure of the SLAM

algorithm was able to get a more accurate reconstruction of the map. In real time, we saw how the map readjusted itself to overlap repeating data. By updating the relevant pointer files, the robot was able to drive around in the new reconstructed map.

5 LAB 6 Technical Approach

Ryosei Tanakamaru

We integrate here two components into our autonomous navigation pipeline: a path planner that computes a collision-free route through the environment, and a pure pursuit controller that executes that route on the racecar. We implemented and compared two planning algorithms, A* and RRT*, and evaluated them on path quality and computation time before selecting A* for the full integration. The planned waypoints are then sent to the pure pursuit controller who sends the drive commands.

5.1 Path Planning

5.1.1 Map Initialization

Ryosei Tanakamaru

Upon initialization, the path planner receives an occupancy grid map. Each cell in the grid encodes whether the corresponding region of the environment is occupied, free, or unknown. Before any path search is performed, the map undergoes an *obstacle inflation* step, where every occupied cell is expanded outward by the robot's radius of 0.3m. This grows each obstacle by that margin in all directions. Figure 10 shows the before and after of map inflation.

The motivation is to reduce the planning problem to one involving a *point robot*. Because the racecar has finite radius, a path that merely avoids obstacle cells in the raw map may still result in collisions between the vehicle and a wall. By pre-inflating obstacles by the robot radius, any path that the planner finds through the inflated map is guaranteed to be safe. This also simplifies the search: the planner only needs to consider a single point traversing a grid, rather than accounting for the vehicle's geometry at every configuration.

5.1.2 A*

Ryosei Tanakamaru

A* is a graph-based search algorithm that finds the shortest path between a start node and a goal node by maintaining a priority queue of candidate nodes ordered by a cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost to reach node n from the start and $h(n)$ is a heuristic estimate of the remaining cost to the goal. By combining accumulated cost with a forward-looking estimate, A* is guaranteed to find the optimal path if the heuristic is admissible, meaning it never overestimates the true remaining cost. We use the euclidean distance to

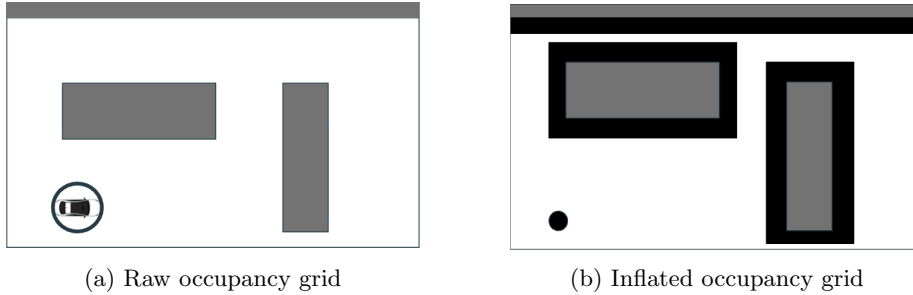


Figure 10: Occupancy grid before and after obstacle inflation by the robot radius of 0.3 m.

the goal as our heuristic, which is admissible on a grid where diagonal movement is permitted.

To improve the quality and smoothness of the path, we apply a line-of-sight pruning step to the raw A* search trajectory. The raw path consists of a sequence of adjacent grid cells, which tends to produce unnecessarily jagged trajectories. Our pruning algorithm iterates through the way-point sequence and for each point, checks whether a straight line to a later point in the sequence is free of obstacles in the inflated map. If the intermediate way-points between two nodes are all "visible" to one another, the intermediate nodes are removed and the two endpoints are connected directly. This process repeats until no further way-points can be eliminated, as you can see in Figure 11 This produces a compact, smooth path that is much easier for the pure pursuit controller to follow without noisy oscillation.

5.1.3 RRT*

Matthew Kim

Rapidly-exploring Random Trees (RRT) is a sampling-based algorithm used for path planning, particularly in high-dimensional spaces with obstacles. RRT quickly finds a feasible path by randomly growing a tree, while RRT* is an extension that provides asymptotic optimality, meaning it converges to the shortest path over time. RRT starts with a tree containing only the start node; from there it randomly samples a point in the map, finds the nearest nodes in the existing tree, and takes a step towards the sample to create a new node. If the new edge collides with an obstacle, the algorithm discards the new node. This overall procedure results in a valid but "jagged" or non-optimal path quickly.

RRT* is an improved version of RRT that minimizes the total path cost (distance) by building upon RRT but adds two key features. First is a neighbor search or lowest cost parent. Instead of connecting a new node only to the nearest neighbor, RRT* checks for other nearby nodes within a radius. It then connects the new node to the neighbor that results in the lowest cumulative cost

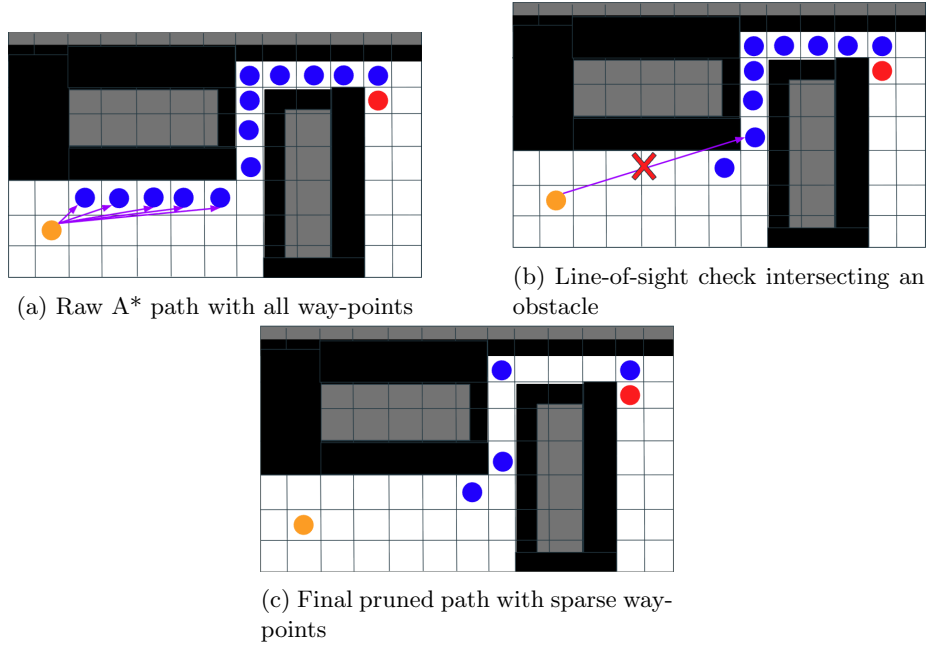


Figure 11: Way-point pruning pipeline: the dense A* output is pruned by iteratively removing intermediate way-points where line-of-sight to a later point is unobstructed.

from the start. Second, it rewires the tree by checking if connecting neighboring nodes through the new node results in a shorter path after adding a new node. If this condition is met, it updates the parent-child relationships, essentially optimizing the tree structure over time. In conclusion, RRT* finds an asymptotically optimal path instead of just a feasible path, which results in a smooth, direct, and shorter path. It is slower due to the extra calculations, especially coupled with how many iterations will be taken. Figure 12 below shows how RRT* would generate a tree and select the optimal path.

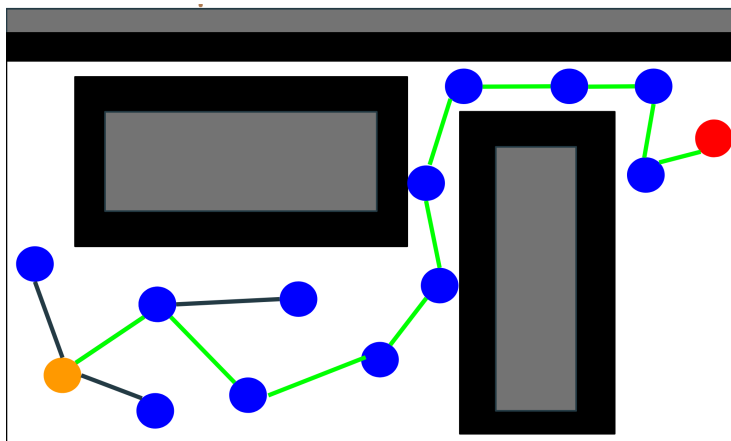


Figure 12: RRT* Diagram

5.1.4 Path Planning Analysis

Matthew Kim

Before moving onto integrating this with the pure pursuit controller, we analyzed the A* and RRT* algorithms to see which would be more optimal. We compared the average path efficiency based on the length of the actual path to the absolute Euclidean distance of the start and goal points, and we also compared how long each algorithm took to compute. Figure 13 shows the generated paths for the two algorithms.

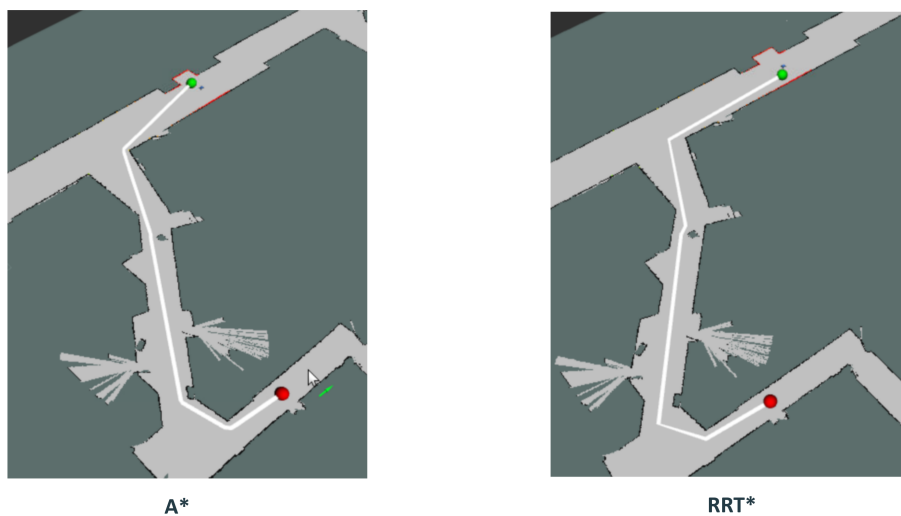


Figure 13: A* and RRT* Published Paths

We analyzed these metrics for various input metrics: for RRT* we varied the number of iterations at 7000, 10000, and 15000 iterations and for the A*, we adjusted the heuristic weight at $\epsilon = 1$ and $\epsilon = 0.7$. As expected, we see a nonlinear positive correlation between the number of iterations and the computation time with RRT* in Figure 15. Interestingly, RRT* had a vast range of computation times as seen with the raw data in Figure 14 where it even outperformed A* at various times. RRT* tended to be very slightly more efficient in its path planning distance efficiency as seen in Figure 16. Given the minute difference in path efficiency compared to the time efficiency, we opted to use A* for our full integration for Lab 6.

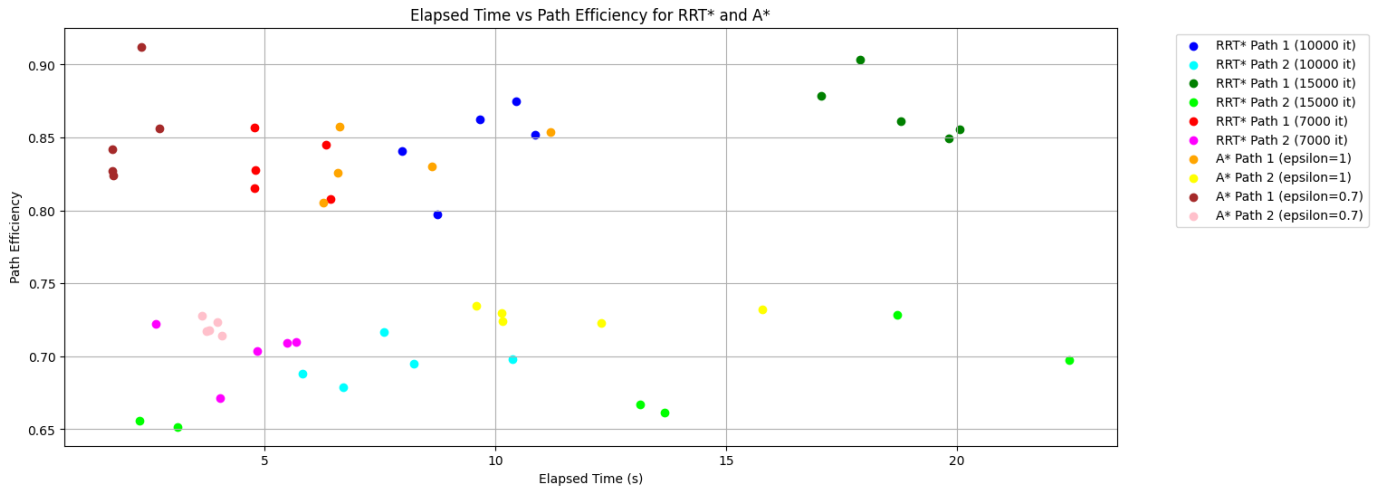


Figure 14: Elapsed Time vs Path Length Ratio (Compared to Straight Line)

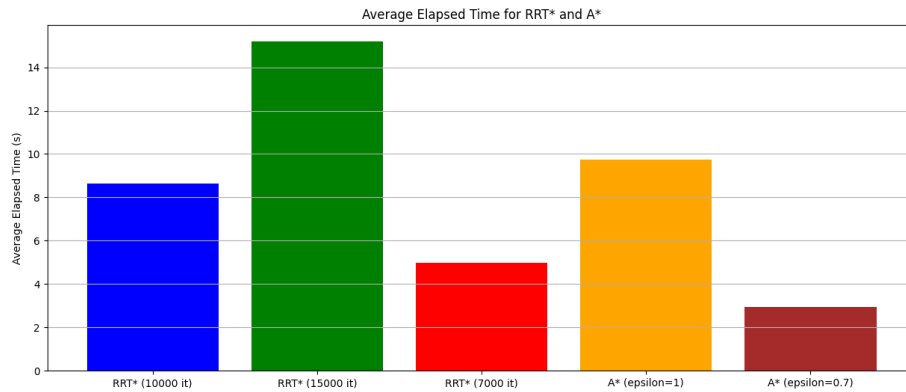


Figure 15: Path Planner Average Elapsed Time

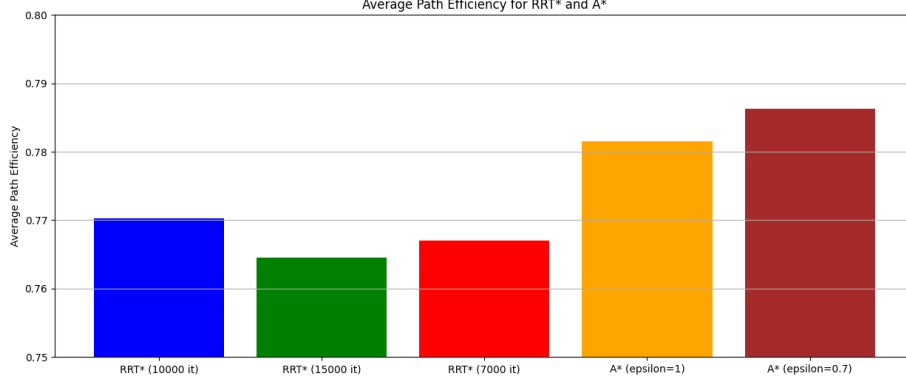


Figure 16: Path Planner Path Length Ratio (Compared to Straight Line)

5.2 Pure Pursuit

J. Desire Abayo

5.2.1 Controller

As shown in Figure 17, pure pursuit steers the car by computing the angle η between the car's current heading and a lookahead point on the planned trajectory. The goal point is first transformed from the map frame into the car's local frame via a standard rotation:

$$\begin{aligned} x &= \cos(\psi) \Delta x + \sin(\psi) \Delta y \\ y &= -\sin(\psi) \Delta x + \cos(\psi) \Delta y \end{aligned}$$

where ψ is the car's yaw. The heading error is then $\eta = \text{atan2}(y, x)$. Treating the car, the goal point, and the instantaneous turn center as an isosceles triangle yields the curvature $\kappa = 1/R = 2 \sin(\eta)/L_1 = 2y/L_1^2$, and the steering command follows as

$$\delta = \arctan\left(\frac{2L \sin \eta}{L_1}\right) = \arctan(\kappa \cdot L_{\text{wheelbase}}) \quad (9)$$

where $L_{\text{wheelbase}} = 0.325$ m and L_1 is the lookahead distance.

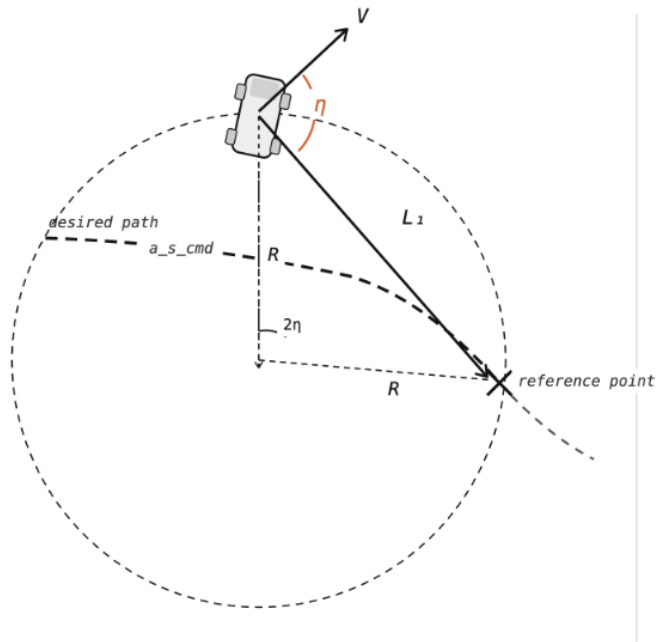


Figure 17: Pure Pursuit computes steering angle from car heading and lookahead point geometry

5.2.2 Results

We evaluated three lookahead distances — $L_1 \in \{0.5, 1.0, 2.0\}$ m — on the same trajectory in the Stata basement map. As visible in Figures 18, smaller lookahead produces tighter corner tracking while larger values cause the car to cut corners. This is confirmed quantitatively by the cross-track error: mean errors of 0.036 m, 0.083 m, and 0.144 m for $L_1 = 0.5$, 1.0, and 2.0 m respectively, with standard deviations of 0.152 m, 0.203 m, and 0.283 m. We selected $L_1 = 0.5$ m as our operating configuration, as it achieves the lowest deviation from the planned path.

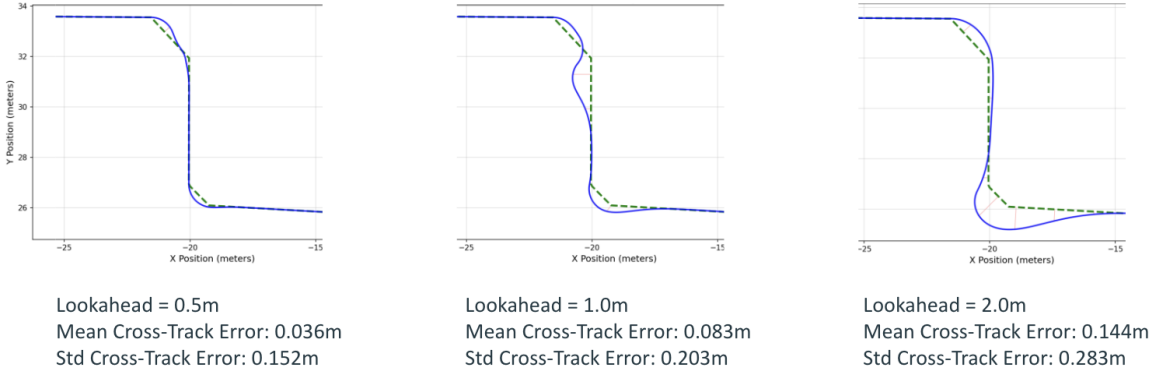


Figure 18: Pure Pursuit cross-track error confirms path deviation grows with lookahead

6 LAB 6 Experimental Results

Franklin Chen

After implementing the path planners and path follower, the programs were integrated into the full localization pipeline and launch files were created to enable proper communication between modules. Experiments were conducted in two phases: simulation followed by real-world validation.

6.1 Simulation Results

Initial experiments were performed in simulation with various path around the map to validate the integration. Figure 19 shows a representative run from the simulated environment.

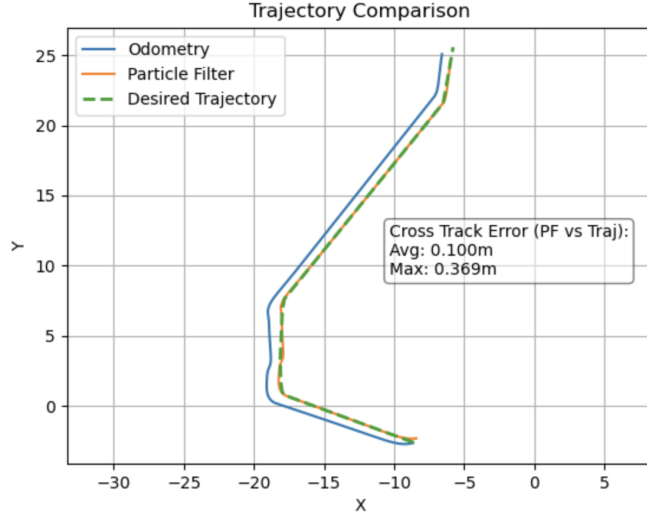


Figure 19: Full Integration in Simulation Example

As shown in Figure 19, the vehicle successfully followed the planned path with high precision. The followed trajectory, however, exhibits a small, consistent cross-track error from the desired path, which remained relatively constant throughout the run. This behavior suggests a slight offset of the estimated location of the vehicle from its actual position in the localization process, rather than from path-following performance.

6.2 Real-World Results

Following the successful simulation testing, experiments were conducted in physical environment to further evaluate performance. Figure 20 presents results from one of these trials.

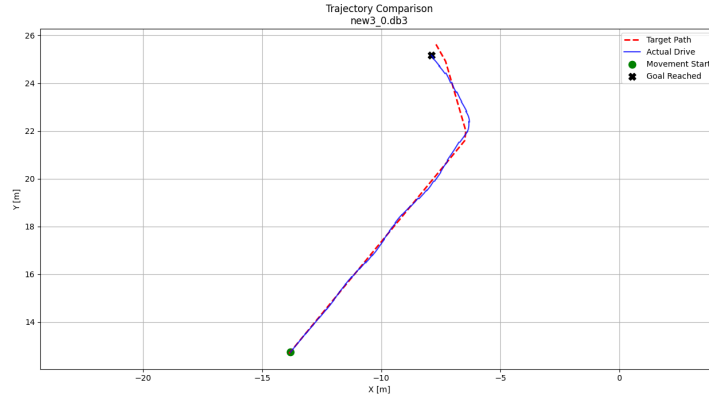


Figure 20: Full Integration in Real Life Example

The real-world experimentation included additional challenges compared to simulation. As seen in Figure 20, the vehicle experienced greater difficulty maintaining the desired trajectory, with increased deviation toward the end of the path. These additional errors could be attributed to noise in measurement, imperfect controls, and difference between the internal and real-world map. Despite these factors, the vehicle still maintained reasonable path following and shows areas for possible improvement.

7 Conclusion

Franklin Chen (Lab 6), Matthew Kim (Lab 5)

Based on both the simulation and real-life experimental results, both path planners display the capability to compute efficient paths in a timely manner. Additionally, the pure pursuit algorithm, given any path, can follow the planned trajectory with minimal error. The path planner and follower are able to be integrated into the localization pipeline well and produce movement to any desired location safely.

In the future, further testing on RRT* and A* with longer trajectory can assist with determining which model to use depending on conditions and requirements. Based on the architecture of the two models, RRT* is suspected to start being more efficient at the increased distance. Additional logic can also be integrated into the path following program to vary look-ahead length depending on different speeds and environment conditions. This would be especially helpful for the final challenge and other situations involving high speeds or difficult paths.

8 Lessons Learned

Ryosei Tanakamaru

Because this lab built directly on our Monte Carlo Localization implementation from lab 5, the accuracy of our pose estimates had an immediate impact on path planning performance. This dependency made clear that in complex, multi-component systems, the quality of each subsystem sets a hard ceiling on the performance of everything built on top of it.

J Desire Abayo

A shorter lookahead isn't what I was expecting initially. And I realized that debugging path following is way easier when you visualize both the planned and actual path side by side in RViz.

Franklin Chen Compared to previous labs, this is the first lab that directly built off another one. This introduces additional factors to consider when debugging and fine-tuning. The lab also stressed the importance of proper implementation and integration of different modules to avoid compounding issues.

Matthew Kim I learned about optimizing algorithms for A* and RRT* in depth and how they work at a deeper level. As always, I also learned more about how to integrate various components better in a systems level project and how to work better as a team member.